Baseline Bots and Evaluation Metrics

1 Purpose of this Note

The goal of Week 1 was not to build a strong poker AI immediately. The goal was to build a clean experimental pipeline:

1. load a poker game from OpenSpiel,
2. create simple baseline agents,
3. make them play against each other,
4. record payoffs in a reproducible way,
5. produce metrics that can later be used to compare CFR, CFR+, and MCCFR agents.

The five Week 1 bots are intentionally simple. They act as "calibration points" for the project. Before we claim that a CFR-style bot is good, we need to show that it beats random play, degenerate fixed strategies, and simple heuristic strategies.

2 Game Used in Week 1

The Week 1 tournament uses `kuhn_poker` from OpenSpiel.

OpenSpiel provides the game rules: card dealing, legal actions, state transitions, terminal states, and final payoffs. Our agents do not implement the Kuhn Poker rules directly. Instead, at every decision point, OpenSpiel tells us which actions are legal, and our bot chooses one of those legal actions.

For Kuhn Poker, OpenSpiel uses the following card encoding:

OpenSpiel card id	card name
0	Jack
1	Queen
2	King

The legal player actions are encoded as:

Action id	Meaning in Kuhn Poker
0	Pass / fold depending on context
1	Bet / call depending on context

The meaning of an action id depends on the current history. For example, action `1` can mean "bet" if no bet has been made yet, but it can mean "call" if the opponent has already bet.

3 Why Start with Baseline Bots?

If we begin directly with CFR or MCCFR, it becomes hard to tell whether the evaluation setup itself is correct. Baseline bots serve four purposes:

1. **Sanity check**: Random vs. Random should be roughly symmetric.
2. **Debugging**: Degenerate agents like Always-Fold and Always-Call make bugs easier to see.
3. **Reference points**: Later, CFR agents should clearly beat these simple baselines.
4. **Narrative**: The project can be explained as a progression from simple behavior to principled regret minimization.

4 The Five Week 1 Bots

4.1 Bot 1: Random Agent

Core idea

The Random Agent chooses uniformly among the legal actions returned by OpenSpiel.

```
class RandomAgent(Agent):
    name = "random"

    def __init__(self, seed: int | None = None):
        self.rng = np.random.default_rng(seed)

    def act(self, state):
        legal = state.legal_actions()
        return int(self.rng.choice(legal))
```

Why we chose it

Random play is the lowest meaningful baseline. If an agent cannot beat Random, it is not useful. It also tests whether the environment and legal-action interface are working correctly.

What it teaches us

It teaches us the floor of performance. Random play has no poker knowledge and no strategic reasoning. Any domain-aware or equilibrium-aware bot should beat it over enough hands.

4.2 Bot 2: Always-Call Agent

Core idea

The Always-Call Agent chooses the aggressive/continuing action whenever possible. In Kuhn Poker, this often means betting or calling, depending on the state.

Why we chose it

Always-Call is a degenerate fixed strategy. It is simple, predictable, and exploitable. This makes it useful for checking whether other bots can exploit obvious strategic weaknesses.

What it teaches us

It shows that a bot can perform well by never folding against very weak opponents, but it can also be punished by more intelligent strategies. It is not an equilibrium strategy.

4.3 Bot 3: Always-Fold Agent

Core idea

The Always-Fold Agent chooses the passive/folding action whenever possible.

Why we chose it

Always-Fold is another degenerate fixed strategy. It gives us a clear example of a very exploitable bot. If a bot cannot beat Always-Fold, the evaluation code is probably wrong.

What it teaches us

It demonstrates why blindly giving up is bad in poker. Against Always-Fold, active opponents can win many pots without needing strong cards.

4.4 Bot 4: Rule-Based Agent

Core idea

The Rule-Based Agent uses a small hand-written decision rule. For example, it may act more aggressively with stronger private cards and more passively with weaker private cards.

Why we chose it

This is the first bot with poker-specific logic. It is not learning; it is simply using human-coded heuristics. We chose it because it sits between trivial fixed strategies and more principled algorithms.

What it teaches us

It tests whether basic domain knowledge improves performance. If a simple rule-based strategy beats Random, that confirms that the environment is sensitive to strategic quality.

4.5 Bot 5: EV-Heuristic Agent

Core idea

The EV-Heuristic Agent tries to make decisions using an expected-value-inspired rule. Instead of simply following a fixed action pattern, it attempts to choose actions that look profitable under a simplified model of the opponent or game state.

Why we chose it

This is the most "principled" Week 1 baseline. It is still not CFR, but it introduces the idea that actions should be chosen by comparing expected payoffs.

What it teaches us

It bridges the gap between hand-coded heuristics and real game-theoretic learning. Later, CFR will replace this simplified EV reasoning with counterfactual regret minimization over information sets.

5 Summary Table of Bots

Bot	Decision rule	Why included	Expected role
Random	Uniformly samples from legal actions.	Establishes the lowest nontrivial baseline.	Floor of skill.
Always-Call	Calls/checks/bets when possible depending on action encoding.	Tests a simple, predictable active strategy.	Degenerate aggressive baseline.
Always-Fold	Folds/passes when possible.	Tests whether other bots exploit obvious weakness.	Degenerate passive baseline.
Rule-Based	Uses hand-coded poker logic.	Introduces domain knowledge without learning.	Human heuristic baseline.
EV-Heuristic	Uses expected-value-inspired decisions.	Introduces payoff-based reasoning before CFR.	Strongest simple baseline / bridge to CFR.

6 Tournament Setup

The tournament is a round-robin among the five agents. Every pair of agents is evaluated. With five agents, the number of unordered matchups is:

$$\binom{5}{2} = 10.$$

For each matchup, we run multiple duplicate pairs. A duplicate pair means:

1. Play one hand with Agent A as Player 0 and Agent B as Player 1.
2. Then replay the same card-deal seed with seats swapped: Agent B as Player 0 and Agent A as Player 1.
3. Average Agent A's payoff across the two seat positions.

This reduces card-luck and seat-position variance. Poker outcomes are noisy, so duplicate evaluation gives a fairer estimate of the strategic difference between two agents.

7 Seeds and Randomness

A seed fixes a random number generator so that an experiment is reproducible. In this project, there are two kinds of randomness:

1. **Game/chance randomness**: card deals sampled from OpenSpiel chance outcomes.
2. **Agent randomness**: stochastic choices made by bots like Random Agent or EV-Heuristic Agent.

The card-deal seed controls chance events in `play_one_hand`. Conceptually:

```
rng = np.random.default_rng(deal_seed)
action = rng.choice(actions, p=probs)
```

OpenSpiel provides the possible chance outcomes and their probabilities. NumPy's random number generator samples one of them reproducibly.

For multiple experimental seeds, we use separate blocks of deal seeds. For example, with `n_pairs = 10000`:

Experimental seed	Deal seed block
0	0 to 9,999
1	1,000,000 to 1,009,999
2	2,000,000 to 2,009,999
3	3,000,000 to 3,009,999
4	4,000,000 to 4,009,999

This ensures that the random card-deal sequences for different seeds do not overlap.

8 Metrics in the Result CSV

The output file `results/tables/exp01_baseline_tournament.csv` contains one row per matchup per seed. The main columns are explained below.

8.1 game

`game` tells us which OpenSpiel game was used. In Week 1 this is:

```
kuhn_poker
```

Later, we will also use `leduc_poker`.

8.2 seed

`seed` identifies the experimental repetition. If the row has `seed = 2`, it means that row came from the third independent seeded run of that matchup.

A seed is not a single hand. It initializes a full reproducible sequence of hands.

8.3 agent_a and agent_b

These columns tell us which two bots are being compared. The reported payoff is always from the perspective of `agent_a`.

For example:

```
agent_a = random
agent_b = always_call
mean_payoff_to_a = -0.382
```

This means Random lost approximately 0.382 chips per duplicate pair on average against Always-Call.

8.4 n_duplicate_pairs

`n_duplicate_pairs` is the number of duplicate pairs used for that matchup and seed.

If:

$$\texttt{n\_duplicate\_pairs} = 10000,$$

then the number of actual hands played is:

$$2 \times 10000 = 20000.$$

This is because each duplicate pair contains two hands: one original seat assignment and one swapped-seat replay.

8.5 mean_payoff_to_a

This is the most important Week 1 metric. It measures the average payoff to Agent A:

$$\bar{u}_A = \frac{1}{N} \sum_{i=1}^{N} u_A^{(i)},$$

where N is the number of duplicate pairs and $u_A^{(i)}$ is Agent A's averaged payoff in duplicate pair i.

Interpretation:

Value	Meaning
$\bar{u}_A > 0$	Agent A beats Agent B on average.
$\bar{u}_A < 0$	Agent A loses to Agent B on average.
$\bar{u}_A \approx 0$	Matchup is roughly even.

This is more important than win rate because poker pots can have different sizes. A bot can win many small pots but lose fewer large pots and still have negative expected payoff.

8.6 ci_low and ci_high

These are the lower and upper endpoints of the approximate 95% confidence interval around the mean payoff.

A typical formula is:

$$\bar{u}_A \pm 1.96 \cdot \frac{s}{\sqrt{N}},$$

where:

- $\bar{u}_A$ is the sample mean payoff,
- s is the sample standard deviation of payoffs,
- N is the number of duplicate-pair samples.

Interpretation:

- If the entire interval is above 0, Agent A is reliably better in this experiment.
- If the entire interval is below 0, Agent A is reliably worse.
- If the interval contains 0, the result is not clearly different from an even matchup.

8.7 std

`std` is the standard deviation of duplicate-pair payoffs. It measures how noisy the matchup is.

A high standard deviation means individual hands/pairs vary a lot. A low standard deviation means results are consistent across duplicate pairs.

Some deterministic matchups can have `std = 0`. For example, if the same outcome happens every time, there is no variation.

8.8 win_rate_a

`win_rate_a` is the fraction of duplicate-pair samples where Agent A had positive payoff:

$$\text{win rate}_A = \frac{\#\{i : u_A^{(i)} > 0\}}{N}.$$

This is useful but secondary. It should not be the main metric in poker because not all wins are equally valuable. Winning one large pot may matter more than winning several tiny pots.

8.9 p_value_vs_zero

`p_value_vs_zero` tests whether the mean payoff is statistically different from 0.

Informally:

- A small p-value suggests Agent A's average payoff is not just random noise.
- A large p-value suggests the result could plausibly be close to 0.

For deterministic or zero-variance matchups, this value can become `NaN`. That is not necessarily an error. It simply means the statistical test is not meaningful when there is no variation.

9 Metrics We Will Use Later

Week 1 mainly uses head-to-head payoff metrics. Later, for CFR and MCCFR agents, we will also use game-theoretic metrics.

9.1 Exploitability

Exploitability measures how far a strategy is from Nash equilibrium in a two-player zero-sum game. A Nash equilibrium has exploitability 0.

Interpretation:

- High exploitability: the strategy can be strongly exploited by a best response.
- Low exploitability: the strategy is closer to equilibrium.
- Zero exploitability: Nash equilibrium in the ideal case.

This becomes a headline metric once we implement CFR, CFR+, and MCCFR.

9.2 Regret Curves

For regret-minimization algorithms, we will track average regret over iterations. If regret decreases toward 0, the algorithm is learning a stable strategy.

A typical plot is:

$$\frac{R_T}{T} \quad \text{vs.} \quad T,$$

usually on log-log axes.

10 How to Interpret One Result Row

Suppose a row says:

Column	Value
`game`	`kuhn_poker`
`agent_a`	`random`
`agent_b`	`always_call`
`n_duplicate_pairs`	`10000`
`mean_payoff_to_a`	`-0.382`
`win_rate_a`	`0.2485`

This means:

> Random was evaluated against Always-Call on Kuhn Poker. For this row, 10,000 duplicate pairs were used, corresponding to 20,000 actual hands. The average payoff to Random was negative, so Random lost to Always-Call on average. Its win rate was about 24.85%, but the main metric is the mean payoff, not the win rate.

11 Why Mean Payoff is the Main Metric

Poker is not like a simple win/loss game where every win has the same value. The payoff magnitude matters. Consider two hypothetical bots:

- Bot X wins 80 small pots of 1 chip each and loses 20 large pots of 10 chips each.
- Bot Y wins only 40 hands but wins large pots and loses small pots.

Bot X has a high win rate but negative total payoff:

$$80 \cdot 1 - 20 \cdot 10 = 80 - 200 = -120.$$

So win rate alone can be misleading. Mean payoff directly measures how much the bot wins or loses in chip units.

12 Final Takeaway

The five Week 1 bots are not meant to solve poker. They are there to test the machinery:

- OpenSpiel correctly runs Kuhn Poker.
- Agents choose among legal actions.
- The tournament loop can compare agents fairly.
- Duplicate-pair evaluation reduces card-luck variance.
- Mean payoff, confidence intervals, standard deviation, win rate, and p-values provide a clear first evaluation layer.

Once this baseline layer is correct, the project can move to regret matching, vanilla CFR, CFR+, and MCCFR with confidence that the environment and evaluation pipeline are working.